

Popravka.ba - Strukturalni dizajn paterni

1 Adapter

Adapter je dizajn patern koji služi kako bi mogli koristiti već postojeće klase u situacijama kada njihov interfejs ne odgovara u određenoj upotrebi, bez izmjene koda originalnih klasa. U našem sistemu, Adapter dizajn patern možemo koristiti na više mjesta:

- **Email servis:** Klase EmailVerifikacijaFirme i EmailNotifikacijaOglas ovise o IEmailSender interfejsu. Pošto postoje različiti provajderi za slanje notifikacija, kao i za slanje potrebnih kredencijala za potvrdu računa putem e-maila, čiji se načini rada međusobno razlikuju, uvodimo adapter klasu IEmailSenderAdapter. IEmailSender je interfejs koji u sebi ima definisane metode za slanje e-mailova. Pomoću adaptera IEmailSenderAdapter, mi prilagođavamo interfejs vanjskog sistema za slanje e-mailova prema interfejsu kojeg smo mi definisali u sistemu.
- **Autentikacija:** Klasa IdentityUser ima u sebi atribut "emailConfirmed", koji služi za provjeru verifikacije računa. Uveli smo interface IAuthenticationService, koji u sebi ima definisane metode za autentikaciju računa. Pošto se verifikacija računa može vršiti putem vanjskih servisa, koji se međusobno razlikuju po načinu rada i implementaciji, uvodimo adapter AuthenticationAdapter. Ovaj adapter vrši prilagođavanje interfejsa vanjskih servisa za autentikaciju računa prema interfejsu koji je definisan u sistemu.
- Uvođenjem ovih adaptera, naš sistem postaje neovisan o konkretnom e-mail provajderu, odnosno o servisu za autentikaciju korisnika, te omogućava jednostavnu nadogradnju i održavanje našeg sistema, bez promjene već postojećeg koda.

2 Façade

Façade je dizajn patern koji se koristi kada želimo od korisnika da sakrijemo složenost sistema, te da mu na jednostavan način omogućimo korištenje usluga sistema. Façade predstavlja interfejs koji olakšava korištenje sistema i koji u sebi sadrži metode i funkcionalnosti sistema koje može korisnik da koristi, bez prikaza implementacije sistema. U našem slučaju, klasa Oglas se dijeli na više dijelova, OglasRadnoMjesto, OglasUsluge, OglasMajstor. Kreiranjem façade OglasFacade, mi korisniku na jednom mjestu i na jednostavan način pružamo upotrebu svih funkcionalnosti sistema koje mu se nude, bez njegovog zalaska u ostale dijelove koda.

3 Decorator

Decorator je dizajn patern koji se koristi kada želimo da dinamički dodajemo nove odgovornosti i ponašanja objektu, bez mijenjanja izvornog koda. Decorator možemo koristiti za dodavanje novih funkcionalnosti, na osnovu uloge korisnika u sistemu.

U našem sistemu, Decorator dizajn patern možemo koristiti za razdvajanje funkcionalnosti sistema u ovisnosti od uloge koju taj korisnik ima. Definiranjem interfejsa `IApplicationUser`, koja u sebi ima osnovne funkcionalnosti korisnika sistema, zatim implementiranjem klase `ApplicationUser`, te na kraju implementiranjem baznog dekoratora `KorisnikDekorator`, dobijamo "bazu" koju možemo da proširujemo novim dekoratorima (`KlijentDekorator`, `MajstorDekorator`, `FirmaDekorator`, `AdminDekorator`).

U tim novim dekoratorima dodajemo funkcionalnosti koje su specifične samo za tu ulogu u sistemu (npr. jedino `FirmaDekorator` ima metode za zapošljavanje majstora i prolaska kroz verifikaciju, kreiranje oglasa imaju `KlijentDekorator`, `MajstorDekorator`, `FirmaDekorator` itd.)

4 Bridge

Bridge pattern omogućava odvajanje apstrakcije od njene implementacije kako bi se obje mogle nezavisno mijenjati i proširivati. Koristi se za povezivanje više različitih apstrakcija i implementacija bez stvaranja velikog broja klasa.

5 Composite

Composite obrazac omogućava grupiranje objekata u hijerarhijske strukture kako bi se pojedinačni objekti i grupe tretirali na isti način. U sistemu postoje hijerarhijski odnosi između entiteta koji se mogu iskoristiti primjenom ovog obrasca.

Unutar našeg sistema, Composite obrazac se može koristiti prilikom razmatranja hijerarhijskog odnosa Firma - Majstor. Composite paternom možemo stvoriti kompozitne objekte (Firma), koja u sebi može sadržavati druge entitete ili pojedinačne objekte (Majstor). Odnosno u našem sistemu, firma je sačinjavana od više majstora, koji su u ovom slučaju podobjekti. Bez primjene Composite paternu, kod bi se morao razlikovati, radio on sa jednim majstorom, ili sa firmom koji upravlja grupom majstora.

6 Proxy

Proxy Pattern predstavlja posrednika koji kontroliše pristup stvarnom objektu. Koristi se za upravljanje pristupom objektima, odgađanje njihovog kreiranja, komunikaciju sa udaljenim objektima ili implementaciju sigurnosnih pravila pristupa. Cilj korištenja Proxy objekta je očuvanje integriteta i zaštite privatnosti osjetljivih podataka.

U našem sistemu, Proxy patern se može koristiti na više mjesta:

- **Zaštita osjetljivih podataka:** Uz pomoć Proxy paternu, možemo vršiti autorizaciju korisnika prije pristupa određenom dijelu sistema (podacima ili funkcionalnostima).

- **Keširanje (caching) podataka:** Keširanjem, odnosno skladištenjem podataka koji se često koriste, kao i podatke koji su rezultat skupih operacija (npr. pretraga oglasa ili kreiranje rang liste majstora) možemo ubrzati rad našeg sistema, samim tim i poboljšati user experience.

Što se tiče našeg sistema, mi smo Proxy pattern implementirali u svrhu zaštite podataka i funkcionalnosti našeg sistema. U interfejsu IFirmaOperacije nalaze se metode za koje ne želimo da svako ima pristup njima. Pošto sve firme nisu istog statusa (verifikacije), uvodimo FirmaProxy koji će biti "posrednik" između objekta klase "Firma" i metoda unutar interfejsa IFirmaOperacije. FirmaProxy, čim primi poziv metode od strane korisnika (klijenta), može provjeriti status verifikacije firme (preko atributa statusVerifikacijeID), te na osnovu tog može odobriti ili odbiti izvršavanje pozvane metode. Na ovaj način, samo verifikovane firme mogu izvršavati metode unutar interfejsa, bez ikakve promjene implementacije klase "Firma", te naš sistem postaje lakši za održavanje i proširivanje, jer se kontrola pristupa nalazi na jednom mjestu (FirmaProxy).